

Proyecto integrador — Monitor Web Multiclase de Estaciones

BOC

- **Release:** 3.0

EOC

- **Valor:** 25.00 puntos
- **Modalidad:** grupal
- **Tecnologías:** Node.js, Express, React, Vite, Tailwind CSS, React Router, Prisma ORM, SQLite, JWT, bcrypt, Vitest, Supertest y Playwright.

1. Propósito

Desarrollar una aplicación web de extremo a extremo para administrar varias clases, registrar sus estaciones de trabajo y supervisar su estado desde un panel protegido. Cada estación enviará reportes periódicos (*heartbeats*) y el sistema mostrará si está en estado correcto, advertencia o crítico.

El proyecto toma como referencia funcional el monitor ICMd disponible en los fuentes locales: tabla de clientes, última actualización, tiempo transcurrido, estado por colores, contadores e indicador de conexión. La solución entregada no debe copiar el código Python/Flask existente; debe implementar el problema con la arquitectura y las tecnologías estudiadas en WEB - 01 a WEB - 06 .

2. Resultados de aprendizaje

Al finalizar, el estudiante deberá poder:

1. Diseñar y documentar una API REST con Express y OpenAPI/Swagger.
2. Persistir entidades relacionadas mediante Prisma ORM y SQLite.
3. Construir una SPA modular, responsiva y accesible con React.
4. Consumir una API con `fetch` , `async/await` y `useEffect` .
5. Administrar rutas, estado local y estado global.
6. Autenticar usuarios con bcrypt y JWT y proteger recursos del sistema.
7. Verificar reglas de negocio mediante pruebas unitarias, de integración y E2E.

Correspondencia con el contenido del curso

Unidad	Aplicación en el proyecto
WEB - 01	Arquitectura cliente-servidor, HTTP, Express, rutas REST y Swagger.

Unidad	Aplicación en el proyecto
WEB - 02	Diseño responsivo, formularios, <code>fetch</code> , asincronía, estados de red y CORS selectivo.
WEB - 03	SQLite, Prisma ORM, relaciones, migraciones, CRUD y servidor en capas.
WEB - 04	Vite, componentes, props, <code>useState</code> , renderizado condicional y formularios controlados.
WEB - 05	<code>useEffect</code> , Tailwind CSS, React Router, Context API y persistencia local.
WEB - 06	Vitest, Supertest, Playwright, accesibilidad, bcrypt, JWT y middleware de autorización.

3. Escenario

Un laboratorio atiende varias clases y necesita conservar por separado el estado de las estaciones utilizadas en cada una. Un operador autenticado debe poder crear clases, registrar sus estaciones, cambiar de clase activa, observar cuándo fue recibido el último reporte, identificar condiciones no permitidas y excluir temporalmente una estación de los contadores de su clase.

Para que la evaluación sea reproducible, no es necesario inspeccionar realmente la computadora, la red ni los procesos del sistema operativo. La aplicación incluirá una vista **Simulador** que enviará reportes a la API como si fueran producidos por una estación real.

Toda la información operativa debe sobrevivir al reinicio del servidor. Usuarios, clases, estaciones, exclusiones y reportes se almacenarán en SQLite mediante Prisma; no se permiten arreglos en memoria ni archivos JSON como fuente de datos.

4. Alcance obligatorio

4.1 Autenticación y autorización

- Permitir registrar el primer usuario administrador.
- Permitir iniciar y cerrar sesión.
- Guardar las contraseñas únicamente como hashes de bcrypt.
- Entregar un JWT con expiración máxima de una hora al iniciar sesión.
- Leer el secreto del JWT desde una variable de entorno; nunca incluirlo en el repositorio.
- Proteger el panel, los CRUD de clases y estaciones y el historial mediante middleware.
- Responder `401 Unauthorized` cuando falta el token y `403 Forbidden` cuando el token es inválido o está vencido.
- La interfaz debe redirigir a `/login` cuando no exista una sesión válida.

4.2 Gestión de varias clases

El usuario autenticado podrá crear, listar, consultar, editar y eliminar clases.

Cada clase tendrá como mínimo:

- código único;
- nombre;
- sección;
- aula o ubicación;
- horario descriptivo;
- estado activo/inactivo;
- fecha de creación y fecha de actualización.

La interfaz deberá permitir seleccionar una **clase activa**. El dashboard, los contadores, los filtros, la lista de estaciones, el simulador y el historial mostrarán únicamente datos de esa clase. Cambiar la clase activa debe actualizar la vista sin recargar el navegador y sin mezclar estaciones o reportes de otras clases.

No se podrá crear una clase con código repetido. Al intentar eliminar una clase que contenga estaciones, la API deberá rechazar la operación con **409 Conflict** y explicar que primero deben reasignarse o eliminarse sus estaciones.

4.3 Gestión de estaciones

El usuario autenticado podrá:

- listar estaciones;
- crear una estación;
- consultar su detalle e historial;
- editar nombre, ubicación y estado de exclusión;
- eliminar una estación, solicitando confirmación antes de ejecutar la acción.

Cada estación tendrá como mínimo:

- identificador interno;
- código único dentro de su clase;
- nombre;
- ubicación;
- sistema operativo;
- indicador **ignored** o **ignorada**;
- fecha de creación y fecha de actualización;
- clase a la que pertenece.

Validaciones mínimas:

- código obligatorio, único dentro de la clase y de 3 a 20 caracteres;
- nombre obligatorio y de al menos 3 caracteres;
- ubicación obligatoria;

- sistema operativo seleccionado de una lista cerrada;
- clase existente y activa;
- mensajes de error claros tanto en cliente como en servidor.

Una estación pertenece obligatoriamente a una sola clase. El mismo código de estación no puede repetirse dentro de la misma clase, aunque puede existir en una clase diferente.

4.4 Reportes o *heartbeats*

La vista `/simulador` permitirá seleccionar una estación y enviar un reporte con:

- estado declarado: `OK` , `INTERNET` o `IA` ;
- versión del agente;
- dirección IP simulada;
- uso de CPU entre 0 y 100;
- uso de memoria entre 0 y 100;
- fecha y hora generadas por el servidor.

No se aceptarán reportes de estaciones inexistentes ni porcentajes fuera del rango. Cada reporte válido debe persistirse y actualizar la última actividad de la estación.

4.5 Regla del semáforo

El servidor, y no el componente visual, será la fuente oficial del estado calculado. Se usará la diferencia entre la hora actual y el último reporte:

Estado calculado	Regla
<code>SIN_REPORTES</code>	La estación nunca ha enviado un reporte.
<code>OK</code>	Reporte <code>OK</code> recibido hace 25 segundos o menos.
<code>ADVERTENCIA</code>	Reporte <code>OK</code> recibido hace más de 25 y hasta 40 segundos.
<code>CRITICO</code>	Han pasado más de 40 segundos desde el último reporte.
<code>CRITICO</code>	El último reporte declaró <code>INTERNET</code> o <code>IA</code> , sin importar su antigüedad.
<code>IGNORADA</code>	La estación fue excluida; no participa en los contadores ni alertas.

La función que aplica estas reglas debe ser pura, estar separada del controlador y contar con pruebas unitarias de todos los límites.

4.6 Dashboard

La ruta protegida `/dashboard` deberá mostrar:

- indicador de conexión con la API;
- selector de clase activa y resumen con nombre, sección y horario;

- tarjetas con totales de estaciones activas, correctas, en advertencia, críticas e ignoradas;
- tabla o cuadrícula con código, nombre, ubicación, sistema operativo, última actualización, segundos transcurridos, CPU, memoria y estado;
- colores y texto/iconos para distinguir los estados sin depender únicamente del color;
- filtro por texto y filtro por estado;
- acción para ignorar o reactivar una estación;
- estados visibles de carga, lista vacía y error;
- actualización automática cada cinco segundos usando `useEffect` , `fetch` y su función de limpieza.

No se exige WebSocket. La actualización periódica con `fetch` permite aplicar el contenido visto en clase y simplifica la ejecución de las pruebas.

Los contadores deben calcularse desde los datos persistidos y estar limitados por la clase solicitada. Una estación de otra clase nunca debe modificar el resumen visible.

4.7 SPA, estado global y diseño

- Crear las rutas `/` , `/login` , `/dashboard` , `/clases` , `/clases/:id` , `/estaciones` , `/estaciones/:id` , `/simulador` y una ruta `*` para página no encontrada.
- Navegar internamente con `Link` o `useNavigate` , sin recargar el navegador.
- Separar páginas, componentes, contextos y servicios de API.
- Usar un `AuthContext` para la sesión, un `ClassContext` para la clase activa y un `ThemeContext` para modo claro/oscuro.
- Persistir solamente el tema y los datos necesarios de la sesión en `localStorage` . No usar `localStorage` como sustituto de SQLite.
- Construir formularios controlados y evitar manipulación directa del DOM.
- Mostrar *skeletons* o indicadores equivalentes durante las cargas.
- Aplicar enfoque *mobile-first* y presentar correctamente el sistema en anchos de 375 px y 1280 px.

4.8 Accesibilidad y calidad visual

- Usar HTML semántico, títulos en orden y regiones de navegación y contenido.
- Asociar cada campo con su `label` .
- Permitir completar los flujos principales con teclado.
- Mantener foco visible y contraste legible en ambos temas.
- Anunciar resultados y errores dinámicos con `role="status"` , `role="alert"` o `aria-live` , según corresponda.
- Proporcionar nombre accesible a botones que usen solamente iconos.
- Incluir un título y una descripción adecuados en cada vista principal.

5. Arquitectura mínima

La entrega debe separar cliente y servidor. Se permite agregar archivos cuando tengan una responsabilidad clara.

```
WEB_PROYECTO_NombreApellido/
├── cliente/
│   ├── src/
│   │   ├── components/
│   │   ├── contexts/
│   │   ├── pages/
│   │   ├── services/
│   │   └── App.jsx
│   └── package.json
├── servidor/
│   ├── prisma/
│   │   ├── migrations/
│   │   ├── schema.prisma
│   │   └── seed.js
│   ├── src/
│   │   ├── controllers/
│   │   ├── middleware/
│   │   ├── routes/
│   │   ├── services/
│   │   ├── app.js
│   │   └── server.js
│   ├── tests/
│   │   ├── integration/
│   │   └── unit/
│   └── package.json
├── e2e/
├── .env.example
├── .gitignore
├── playwright.config.js
└── README.md
```

`app.js` debe construir y exportar la aplicación Express. `server.js` será el único archivo que abre el puerto, de modo que Supertest pueda importar la aplicación sin iniciar otro proceso.

6. Modelo de datos mínimo

User

Campo	Restricción
id	Llave primaria.

Campo	Restricción
name	Obligatorio.
email	Obligatorio y único.
passwordHash	Obligatorio; nunca se guarda la contraseña original.
createdAt	Generado automáticamente.

Class

Campo	Restricción
id	Llave primaria.
code	Obligatorio y único.
name	Obligatorio.
section	Obligatorio.
location	Obligatorio.
schedule	Obligatorio.
active	Booleano, valor inicial <code>true</code> .
createdAt , updatedAt	Marcas de tiempo.

Station

Campo	Restricción
id	Llave primaria.
classId	Llave foránea obligatoria hacia <code>Class</code> .
code	Obligatorio y único dentro de su clase.
name	Obligatorio.
location	Obligatorio.
operatingSystem	Obligatorio.
ignored	Booleano, valor inicial <code>false</code> .
createdAt , updatedAt	Marcas de tiempo.

Report

Campo	Restricción
id	Llave primaria.
stationId	Llave foránea hacia Station .
declaredStatus	OK , INTERNET o IA .
agentVersion	Obligatorio.
ipAddress	Obligatorio.
cpuPercent , memoryPercent	Valores entre 0 y 100.
createdAt	Hora asignada por el servidor.

Las relaciones obligatorias son Class 1:N Station y Station 1:N Report . La combinación classId + code debe tener una restricción única. La eliminación de una clase con estaciones será restringida y la eliminación de una estación eliminará sus reportes asociados mediante una regla explícita en el esquema.

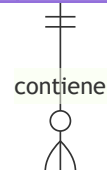
Se deben incluir la migración y un *seed* con al menos dos clases y tres estaciones por clase; por lo menos dos estaciones de cada clase tendrán reportes de ejemplo. Después de detener y volver a iniciar el servidor deben conservarse las clases, estaciones, reportes y marcas de exclusión.

Diagrama entidad-relación

El siguiente diagrama muestra las cuatro tablas del modelo mínimo, sus llaves primarias (PK), llaves foráneas (FK) y campos únicos (UK). Los tipos se expresan con la nomenclatura de Prisma.

User		
Int	id	PK
String	name	
String	email	UK
String	passwordHash	
DateTime	createdAt	

Class		
Int	id	PK
String	code	UK
String	name	
String	section	
String	location	
String	schedule	
Boolean	active	
DateTime	createdAt	
DateTime	updatedAt	



Station			
Int	id	PK	
Int	classId	FK	
String	code		Unico junto con classId
String	name		
String	location		
String	operatingSystem		
Boolean	ignored		
DateTime	createdAt		
DateTime	updatedAt		



Report			
Int	id	PK	

Int	stationId	FK	
String	declaredStatus		OK, INTERNET o IA
String	agentVersion		
String	ipAddress		
Float	cpuPercent		Entre 0 y 100
Float	memoryPercent		Entre 0 y 100
DateTime	createdAt		

- **Clase y estaciones:** una clase puede tener cero o muchas estaciones; cada estación pertenece a exactamente una clase mediante `Station.classId`.
- **Estación y reportes:** una estación puede tener cero o muchos reportes; cada reporte pertenece a exactamente una estación mediante `Report.stationId`.
- **Usuarios:** `User` permite autenticar al administrador. El modelo mínimo no define llaves foráneas entre usuarios y las demás tablas, por lo que aparece sin relaciones en el diagrama.
- **Unicidad compuesta:** `Station` requiere `@@unique([classId, code])`; su campo `code` no es único por sí solo en toda la tabla.
- **Eliminación:** la relación de `Station` hacia `Class` debe usar `onDelete: Restrict`; la de `Report` hacia `Station`, `onDelete: Cascade`.

En la notación del diagrama, `||` significa exactamente uno y `o{` significa cero o muchos. Las exclusiones se almacenan en `Station.ignored`; no requieren una tabla adicional. El último reporte y el estado calculado se obtienen a partir de los datos persistidos y de las reglas de la sección 4.5.

7. Contrato mínimo de la API

Todas las respuestas serán JSON, excepto la interfaz Swagger. Los errores deben usar una forma consistente, por ejemplo:

```
{
  "error": "El código de la estación ya existe"
}
```

Método	Ruta	Resultado principal	Protección
POST	<code>/api/auth/register</code>	Crea el primer usuario (<code>201</code>).	Pública con restricción de primer usuario.
POST	<code>/api/auth/login</code>	Devuelve JWT (<code>200</code>) o credenciales inválidas (<code>401</code>).	Pública.
GET	<code>/api/classes</code>	Lista todas las clases.	JWT.

Método	Ruta	Resultado principal	Protección
POST	/api/classes	Crea una clase (201).	JWT.
GET	/api/classes/:id	Obtiene la clase y su cantidad de estaciones.	JWT.
PUT	/api/classes/:id	Actualiza una clase.	JWT.
DELETE	/api/classes/:id	Elimina una clase vacía o responde 409 .	JWT.
GET	/api/stations?classId=:classId	Lista estaciones de una clase con último reporte y estado calculado.	JWT.
POST	/api/stations	Crea una estación asociada a una clase (201).	JWT.
GET	/api/stations/:id	Obtiene detalle e historial.	JWT.
PUT	/api/stations/:id	Actualiza datos editables.	JWT.
PATCH	/api/stations/:id/ignore	Ignora o reactiva la estación.	JWT.
DELETE	/api/stations/:id	Elimina estación (204).	JWT.
POST	/api/stations/:id/reports	Registra un <i>heartbeat</i> (201).	JWT.
GET	/api/dashboard/summary?classId=:classId	Devuelve los cinco contadores de una clase.	JWT.
GET	/api/health	Informa si la API está disponible.	Pública.
GET	/api-docs	Presenta Swagger UI con todos los endpoints.	Pública.

7.1 Convenciones del contrato

- La URL base es `http://localhost:3000/api` durante el desarrollo.
- Las rutas protegidas reciben `Authorization: Bearer <token>`.
- Los identificadores `:id` son enteros positivos. Un formato inválido responde `400`; un recurso inexistente responde `404`.
- Los cuerpos se envían como `application/json`.
- Las fechas se devuelven en formato ISO 8601 UTC.
- Las respuestas de error mantienen la estructura `{ "error": "mensaje", "details": [] }`; `details` es opcional.
- Ninguna respuesta debe exponer `passwordHash`, el secreto JWT ni información interna de la base de datos.

7.2 Autenticación

POST /api/auth/register

Registra únicamente el primer administrador del sistema.

```
{
  "name": "Ana López",
  "email": "ana@example.com",
  "password": "ClaveSegura123"
}
```

- 201 : devuelve `id`, `name`, `email` y `createdAt` del usuario.
- 400 : falta un campo o el correo/contraseña no cumple la validación.
- 409 : el correo ya existe.
- 403 : el sistema ya tiene un administrador registrado.

POST /api/auth/login

```
{
  "email": "ana@example.com",
  "password": "ClaveSegura123"
}
```

- 200 : devuelve `{ "token": "...", "expiresIn": 3600, "user": { ... } }`.
- 400 : faltan credenciales o tienen formato inválido.
- 401 : correo o contraseña incorrectos, sin indicar cuál falló.

7.3 Clases

GET /api/classes

Acepta el parámetro opcional `active=true|false`. Devuelve un arreglo de clases; cada elemento incluye `stationCount`. Sin resultados devuelve 200 y `[]`.

POST /api/classes

```
{
  "code": "WEB-01",
  "name": "Programación Web",
  "section": "0801",
  "location": "Laboratorio 2",
  "schedule": "Lunes y miércoles 08:00-09:30",
  "active": true
}
```

- 201 : devuelve la clase persistida.
- 400 : datos incompletos o inválidos.

- 409 : código de clase duplicado.

GET /api/classes/:id

- 200 : devuelve la clase, `stationCount` y sus marcas de tiempo.
- 404 : la clase no existe.

PUT /api/classes/:id

Recibe todos los campos editables mostrados en `POST /api/classes`.

- 200 : devuelve la clase actualizada.
- 400 : cuerpo incompleto o inválido.
- 404 : la clase no existe.
- 409 : el nuevo código pertenece a otra clase.

DELETE /api/classes/:id

- 204 : elimina una clase que no contiene estaciones y no devuelve cuerpo.
- 404 : la clase no existe.
- 409 : la clase todavía contiene estaciones.

7.4 Estaciones

GET /api/stations?classId=:classId

`classId` es obligatorio. Se permiten además `status` y `q` como filtros opcionales. Devuelve solamente las estaciones de la clase solicitada. Cada elemento incluye los datos de la estación, `lastReport`, `elapsedSeconds` y `calculatedStatus`.

BOC

Los parámetros y campos de esta operación tienen el siguiente significado:

- `q` (*query* o búsqueda) es un parámetro de consulta opcional que filtra por texto. La API elimina los espacios al inicio y al final y busca una coincidencia parcial, sin distinguir mayúsculas de minúsculas, en `code`, `name` o `location`. Si se omite o queda vacío después de eliminar los espacios, no se aplica el filtro de texto. Por ejemplo, `GET /api/stations?classId=1&q=frontal` encuentra una estación cuyo nombre sea `Estación frontal`. El valor debe codificarse como parte de la URL.
- `lastReport` es el reporte más reciente de la estación, determinado por `Report.createdAt`. Contiene los campos persistidos del modelo `Report`. Su valor es `null` cuando la estación todavía no ha enviado ningún reporte.
- `elapsedSeconds` es la cantidad entera de segundos transcurridos entre la hora actual del servidor y `lastReport.createdAt`. Se calcula al construir cada respuesta, no se almacena en la base de datos, nunca es negativo y usa la parte entera del intervalo. Su valor es `null` cuando `lastReport` es `null`.

- `calculatedStatus` es el estado oficial producido por el servidor al aplicar la regla del semáforo de la sección 4.5. Sus valores posibles son `SIN_REPORTES`, `OK`, `ADVERTENCIA`, `CRITICO` e `IGNORADA`. No es el mismo campo que `lastReport.declaredStatus`: el estado declarado proviene del último reporte, mientras que el calculado también considera su antigüedad y el valor `ignored` de la estación. El filtro `status`, cuando se envía, se aplica sobre `calculatedStatus`.

Ejemplo de un elemento de la respuesta:

```
{
  "id": 7,
  "classId": 1,
  "code": "PC-01",
  "name": "Estación frontal",
  "location": "Fila A",
  "operatingSystem": "WINDOWS",
  "ignored": false,
  "createdAt": "2026-09-18T20:00:00.000Z",
  "updatedAt": "2026-09-18T20:00:00.000Z",
  "lastReport": {
    "id": 42,
    "stationId": 7,
    "declaredStatus": "OK",
    "agentVersion": "1.0.0",
    "ipAddress": "192.168.1.21",
    "cpuPercent": 35,
    "memoryPercent": 62,
    "createdAt": "2026-09-18T20:10:00.000Z"
  },
  "elapsedSeconds": 12,
  "calculatedStatus": "OK"
}
```

LEOC

- `200`: devuelve el arreglo, incluso si está vacío.
- `400`: falta `classId` o algún filtro es inválido.
- `404`: la clase no existe.

POST /api/stations

```
{
  "classId": 1,
  "code": "PC-01",
  "name": "Estación frontal",
  "location": "Fila A",
  "operatingSystem": "WINDOWS"
}
```

- `201` : devuelve la estación persistida con `ignored: false` .
- `400` : datos incompletos, sistema operativo no permitido o clase inactiva.
- `404` : la clase no existe.
- `409` : el código ya existe dentro de esa clase.

GET /api/stations/:id

Acepta `limit` como parámetro opcional para el historial, con valor inicial 20 y máximo 100.

└ BOC

- `limit` indica cuántos reportes recientes se incluyen en `reports` . Debe ser un número entero entre 1 y 100. Si se omite, la API utiliza 20. Solo limita el arreglo del historial; no cambia los datos de la estación, de la clase ni el cálculo del estado. Por ejemplo, `GET /api/stations/7?limit=5` solicita los cinco reportes más recientes de la estación 7.
- **Detalle** significa la representación completa de la estación solicitada: sus campos persistidos, la clase a la que pertenece, `lastReport` , `elapsedSeconds` y `calculatedStatus` . `detalle` no es un parámetro ni un campo literal del JSON.
- **Historial** es el arreglo `reports` formado únicamente por reportes de esa estación, ordenados por `createdAt` del más reciente al más antiguo y reducido al valor de `limit` . Si la estación no tiene reportes, se devuelve `[]` ; la API no genera reportes para completar el límite.

Ejemplo de respuesta con `limit=2` :

```
{
  "station": {
    "id": 7,
    "classId": 1,
    "code": "PC-01",
    "name": "Estación frontal",
    "location": "Fila A",
    "operatingSystem": "WINDOWS",
    "ignored": false,
    "createdAt": "2026-09-18T20:00:00.000Z",
    "updatedAt": "2026-09-18T20:00:00.000Z"
  },
  "class": {
    "id": 1,
    "code": "WEB-01",
    "name": "Programación Web",
    "section": "0801",
    "location": "Laboratorio 2",
    "schedule": "Lunes y miércoles 08:00-09:30",
    "active": true
  },
  "lastReport": {
    "id": 42,
    "stationId": 7,
    "declaredStatus": "OK",
    "agentVersion": "1.0.0",
    "ipAddress": "192.168.1.21",
    "cpuPercent": 35,
    "memoryPercent": 62,
    "createdAt": "2026-09-18T20:10:00.000Z"
  },
  "elapsedSeconds": 12,
  "calculatedStatus": "OK",
  "reports": [
    {
      "id": 42,
      "stationId": 7,
      "declaredStatus": "OK",
      "agentVersion": "1.0.0",
      "ipAddress": "192.168.1.21",
      "cpuPercent": 35,
      "memoryPercent": 62,
      "createdAt": "2026-09-18T20:10:00.000Z"
    },
    {
      "id": 41,
      "stationId": 7,
      "declaredStatus": "INTERNET",
      "agentVersion": "1.0.0",
      "ipAddress": "192.168.1.21",
      "cpuPercent": 31,

```

```

    "memoryPercent": 58,
    "createdAt": "2026-09-18T20:09:00.000Z"
  }
]
}

```

↳ EOC

- 200 : devuelve estación, clase, estado calculado y reportes ordenados del más reciente al más antiguo.
- 400 : limit es inválido.
- 404 : la estación no existe.

PUT /api/stations/:id

Recibe `classId`, `code`, `name`, `location` y `operatingSystem`. Permite reasignar la estación a otra clase activa.

- 200 : devuelve la estación actualizada.
- 400 : datos inválidos o clase de destino inactiva.
- 404 : la estación o clase de destino no existe.
- 409 : el código ya existe en la clase de destino.

PATCH /api/stations/:id/ignore

```

{
  "ignored": true
}

```

- 200 : devuelve `id`, `ignored` y `updatedAt`.
- 400 : ignored no es booleano.
- 404 : la estación no existe.

DELETE /api/stations/:id

- 204 : elimina la estación y sus reportes según la relación configurada.
- 404 : la estación no existe.

7.5 Reportes

POST /api/stations/:id/reports

↳ BOC

Un *heartbeat* o «latido» es el reporte periódico con el que una estación avisa al servidor que continúa comunicándose y describe su condición actual. En este proyecto no es un campo adicional llamado `heartbeat`: el cuerpo JSON completo de cada solicitud a esta ruta constituye un *heartbeat* y se persiste como un registro `Report`.

La vista `/simulador` reemplaza al agente real y permite enviar el latido de forma manual. Cada solicitud válida crea un reporte nuevo; no actualiza ni reemplaza el anterior. El servidor identifica la estación mediante `:id`, asigna `createdAt`, convierte el nuevo reporte en `lastReport` y vuelve a calcular `elapsedSeconds` y `calculatedStatus`. En el instante de la respuesta, `elapsedSeconds` es `0`.

LEOC

```
{
  "declaredStatus": "OK",
  "agentVersion": "1.0.0",
  "ipAddress": "192.168.1.21",
  "cpuPercent": 35,
  "memoryPercent": 62
}
```

- `201`: devuelve el reporte persistido, `elapsedSeconds: 0` y el `calculatedStatus` resultante.
- `400`: estado desconocido, datos incompletos o porcentajes fuera de 0 a 100.
- `404`: la estación no existe.

La API ignora cualquier `createdAt` enviado por el cliente y asigna la hora del servidor.

7.6 Resumen y salud

GET /api/dashboard/summary?classId=:classId

`classId` es obligatorio. Devuelve contadores exclusivos de la clase:

```
{
  "classId": 1,
  "active": 4,
  "ok": 2,
  "warning": 1,
  "critical": 1,
  "ignored": 1
}
```

- `200`: resumen calculado correctamente.
- `400`: falta `classId` o no es válido.
- `404`: la clase no existe.

GET /api/health

- `200`: devuelve `{ "status": "ok", "timestamp": "..." }`.
- `500`: devuelve `{ "status": "error", "error": "..." }` cuando no puede consultarse la base de datos.

7.7 Documentación OpenAPI/Swagger obligatoria

El contrato se implementará en OpenAPI 3 mediante un archivo `swagger.json`, `swagger.yaml` o anotaciones equivalentes, y se publicará con Swagger UI en `/api-docs`. **Ningún endpoint de las secciones 7.2 a 7.6 puede quedar sin documentar.**

Cada operación de Swagger debe incluir:

- etiqueta y resumen funcional;
- descripción y requisito de autenticación;
- parámetros de ruta y consulta con tipo, obligatoriedad y ejemplo;
- cuerpo JSON con esquema, campos requeridos, restricciones y ejemplo;
- respuesta exitosa y todas las respuestas de error indicadas;
- referencia a esquemas reutilizables para `User`, `Class`, `Station`, `Report`, `DashboardSummary` y `Error`;
- esquema de seguridad `bearerAuth` de tipo HTTP Bearer con formato JWT.

Swagger UI debe permitir autenticarse mediante el botón **Authorize** y ejecutar las rutas protegidas contra el servidor local. La documentación se considera incompleta si solo enumera las rutas o si no coincide con su comportamiento real.

La API debe retornar, según el caso, `200`, `201`, `204`, `400`, `401`, `403`, `404`, `409` y `500`. No se aceptará responder `200` para todos los resultados. CORS debe aceptar solamente el origen configurado para el cliente.

8. Pruebas obligatorias

Las pruebas usarán una base aislada y no modificarán la base de desarrollo.

8.1 Unitarias — Vitest

Como mínimo, probar:

1. `SIN_REPORTES` cuando no existe último reporte.
2. `OK` exactamente a los 25 segundos.
3. `ADVERTENCIA` después de 25 y exactamente a los 40 segundos.
4. `CRITICO` después de 40 segundos.
5. `CRITICO` para reportes `INTERNET` e `IA`.
6. `IGNORADA` y su exclusión de los contadores.
7. rechazo de CPU o memoria fuera de 0 a 100.

8.2 Integración — Vitest y Supertest

Como mínimo, verificar:

1. registro exitoso, datos incompletos y correo duplicado;
2. inicio de sesión válido e inválido;

3. hash almacenado y verificado con `bcrypt.compare()` ;
4. acceso a una ruta protegida sin token, con token inválido y con token válido;
5. CRUD de clases, código duplicado y rechazo al eliminar una clase con estaciones;
6. creación de estaciones en dos clases, código duplicado dentro de una clase y el mismo código aceptado en clases diferentes;
7. registro válido e inválido de un reporte;
8. aislamiento del listado y del resumen por `classId` ;
9. persistencia de clases, estaciones, reportes y exclusiones al crear una nueva instancia de la aplicación sobre la misma base de prueba.

8.3 E2E — Playwright

Implementar al menos cuatro flujos:

1. iniciar sesión, abrir el dashboard y observar los contadores;
2. crear dos clases, cambiar la clase activa y comprobar que sus estaciones y contadores no se mezclan;
3. crear una estación, enviarle un reporte desde el simulador y comprobar el cambio de estado incluso después de recargar la página;
4. intentar iniciar sesión con credenciales inválidas y comprobar el mensaje accesible.

`npm test` y `npm run test:e2e` deben finalizar sin errores desde una instalación limpia.

9. Documentación y entrega

El `README.md` debe contener:

- descripción breve y capturas de escritorio y móvil;
- requisitos previos;
- instalación del cliente y servidor;
- variables requeridas, documentadas en `.env.example` sin secretos reales;
- migración, carga del *seed* y ejecución;
- comandos de pruebas unitarias, de integración y E2E;
- credenciales de demostración o procedimiento para crear el primer usuario;
- explicación de cómo crear, seleccionar y administrar varias clases;
- URL de Swagger;
- decisiones técnicas y limitaciones conocidas.

Entregar `WEB_PROYECTO_NombreApellido.zip` . El archivo no debe contener

`node_modules` , `dist` , `.env` , bases SQLite generadas, `coverage` , `test-results` , `playwright-report` ni archivos del sistema operativo.

10. Rúbrica — 25.00 puntos

#	Criterio verificable	Evidencia requerida	Pts.
1	Arquitectura del proyecto	Cliente y servidor separados; rutas, controladores, servicios, middleware, páginas, componentes y contextos tienen responsabilidades claras; <code>app.js</code> no abre el puerto.	1.00
2	Persistencia relacional	Prisma y SQLite implementan los cuatro modelos, restricciones, relaciones <code>Class 1:N Station 1:N Report</code> , política de eliminación, migración y <i>seed</i> multiclase; los datos sobreviven al reinicio.	2.50
3	Autenticación y seguridad	Registro controlado, bcrypt, login, JWT desde variable de entorno, expiración y respuestas <code>401 / 403</code> correctas.	2.50
4	API REST y validaciones	CRUD completo de clases y estaciones, filtros por clase, códigos HTTP adecuados, validación en servidor, errores JSON consistentes y CORS selectivo.	3.00
5	Reportes y reglas de estado	El reporte persiste; la API calcula correctamente los seis casos del semáforo y excluye estaciones ignoradas de contadores.	3.00
6	React SPA y navegación	Existen todas las rutas, vistas de administración de clases, vista protegida, página 404, navegación sin recarga y formularios controlados organizados en componentes.	2.00
7	Dashboard e integración multiclase	<code>fetch</code> consume datos reales, permite cambiar la clase activa sin mezclar datos, presenta todas las columnas y contadores, filtros, estados de carga/error/vacío y actualiza cada cinco segundos con <i>cleanup</i> .	2.50
8	Diseño responsivo	Tailwind, enfoque <i>mobile-first</i> , temas claro/oscuro, jerarquía visual coherente y funcionamiento a 375 px y 1280 px.	1.50
9	Estado global y persistencia local	<code>AuthContext</code> , <code>ClassContext</code> y <code>ThemeContext</code> funcionan; la clase activa, el tema y la sesión necesaria sobreviven una recarga sin sustituir la base de datos.	1.00
10	OpenAPI y salud del sistema	Swagger documenta esquemas, seguridad, parámetros y respuestas de todos los endpoints; <code>/api/health</code> alimenta el indicador de conexión.	1.00
11	Pruebas automatizadas	Se incluyen y aprueban todos los casos mínimos unitarios, de integración y los cuatro flujos E2E, incluyendo aislamiento y persistencia multiclase, con una base de prueba separada.	3.00
12	Accesibilidad	Etiquetas, teclado, foco, contraste, HTML semántico y anuncios accesibles se verifican durante los flujos principales.	1.00
13	Documentación y entrega reproducible	<code>README.md</code> , <code>.env.example</code> , scripts, capturas y ZIP cumplen lo solicitado y el proyecto funciona desde una instalación limpia.	1.00
	Total		25.00

Escala por criterio

Cada criterio se calificará según el nivel alcanzado:

- **100 %:** cumple todas las verificaciones y puede demostrarse ejecutando el proyecto o las pruebas.
- **50 %:** cumple al menos la mitad de las verificaciones, pero presenta fallos parciales o evidencia incompleta.
- **0 %:** cumple menos de la mitad, no puede ejecutarse o falta la evidencia.

Reglas especiales:

- El criterio de autenticación recibe **0 puntos** si se almacena alguna contraseña en texto plano o se publica un secreto real.
- El criterio de pruebas recibe **0 puntos** si las pruebas entregadas no pueden ejecutarse o fueron deshabilitadas para aparentar un resultado exitoso.
- Las capturas apoyan la documentación, pero no sustituyen la ejecución del sistema.
- Una funcionalidad que solo existe visualmente y no persiste o no utiliza la API no obtiene los puntos de integración correspondientes.

11. Lista de verificación del estudiante

- ☐ Puedo instalar cliente y servidor siguiendo únicamente el `README.md`.
- ☐ La migración y el `seed` funcionan sobre una base vacía.
- ☐ Puedo reiniciar el servidor sin perder clases, estaciones, reportes o exclusiones.
- ☐ Puedo administrar al menos dos clases y sus datos nunca se mezclan.
- ☐ Ninguna contraseña, secreto o token está incluido en el repositorio.
- ☐ Swagger permite conocer y probar el contrato completo de la API.
- ☐ Todas las rutas protegidas rechazan solicitudes no autorizadas.
- ☐ Los límites de 25 y 40 segundos están cubiertos por pruebas.
- ☐ El dashboard se actualiza y limpia el temporizador correctamente.
- ☐ La aplicación puede operarse con teclado y anuncia los errores.
- ☐ `npm test` y `npm run test:e2e` finalizan en verde.
- ☐ El ZIP tiene el nombre correcto y no contiene artefactos excluidos.

└ BOC

2026-09-18 23:52:44

└ EOC